



UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de la Organización:

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO (UTEQ)

Presenta:

VICTOR GUADALUPE LOPEZ CEBREROS

Grupo:

IDGS17

Unidad 2. Práctica 2. Corrección de Issues en SonarQube

Análisis Estático con SonarQube y Remediación de Issues en un Proyecto ASP.NET Core 10

Resolución integral de hallazgos en las dimensiones Security, Reliability y Maintainability

Todos los valores cuantitativos consignados en este reporte fueron obtenidos directamente de la API REST de SonarQube (<http://localhost:9000>) mediante consulta a los endpoints `/api/measures/component`, `/api/issues/search`, `/api/hotspots/search` y `/api/qualitygates/project_status`. No se trata de estimaciones ni de lecturas visuales del dashboard, sino de las métricas persistidas en la base de datos del servidor tras cada corrida del scanner.

1. Datos generales del análisis

Campo	Valor
Proyecto analizado	MiAppNetCore — ASP.NET Core 10
Project key	MiAppNetCore
Herramienta de análisis	SonarQube Community Edition 26.6.0.123539
Modo de despliegue	Contenedor Docker con persistencia en volúmenes nombrados
Motor de base de datos del servidor	PostgreSQL 15 (contenedor sonar_db)
Cliente de análisis	dotnet-sonarscanner 11.2.1 (herramienta global de .NET)
Motor de análisis subyacente	Analizadores Roslyn (SonarAnalyzer para C#)
Framework de pruebas	xUnit / MSTest sobre MiAppNetCore.Tests
Recolector de cobertura	coverlet.collector, formato OpenCover
Persistencia de la aplicación	SQLite (Microsoft.Data.Sqlite)
Dashboard del proyecto	http://localhost:9000/dashboard?id=MiAppNetCore

Campo	Valor
Análisis inicial (línea base con hallazgos)	2026-08-07, 00:04 hora local (06:04 UTC)
Análisis final (verificación tras remediación)	2026-08-07, 02:10 hora local (08:10 UTC)
Corridas totales del ciclo completo	2 análisis principales más corridas de verificación
Estado final del Quality Gate	OK

2. Metodología del análisis

2.1 Arquitectura del proceso de escaneo

El análisis de proyectos .NET no opera sobre el código fuente de manera aislada, sino que se acopla al propio proceso de compilación. Esta particularidad determina la estructura obligatoria del ciclo, compuesto por cuatro fases ejecutadas en secuencia estricta:

Fase	Comando	Función técnica
Begin	dotnet sonarscanner begin	Descarga el perfil de calidad activo desde el servidor, inyecta los analizadores de Roslyn en el pipeline de MSBuild mediante la modificación de las propiedades de compilación, y prepara el directorio .sonarqube donde se depositarán los resultados intermedios en formato protobuf.
Build	dotnet build --no-incremental	Compila la solución. Durante esta fase los analizadores se ejecutan como parte del compilador de C#, operando sobre el árbol sintáctico (SyntaxTree) y el modelo semántico (SemanticModel) que Roslyn ya construye para la compilación ordinaria.
Test	dotnet test --collect:"XPlat Code Coverage"	Ejecuta la suite de pruebas instrumentando los ensamblados con coverlet para producir el reporte de cobertura en formato OpenCover.

Fase	Comando	Función técnica
End	dotnet sonarscanner end	Consolida los resultados intermedios, los correlaciona con los reportes de cobertura y de resultados de prueba, y transmite el conjunto al servidor mediante una tarea de background que este procesa de forma asíncrona.

La utilización del modelo semántico de Roslyn constituye la diferencia fundamental entre este tipo de análisis y las herramientas basadas en coincidencia de patrones textuales o expresiones regulares. El analizador no busca cadenas de texto: resuelve tipos, ámbitos de variables, sobrecargas de métodos y flujos de datos entre asignaciones, lo que le permite determinar, por ejemplo, que una variable declarada nunca se lee en ninguna ruta de ejecución alcanzable, o que un valor de tipo referencia anulable se dereferencia sin verificación previa.

2.2 Justificación de la compilación no incremental

La bandera --no-incremental resulta indispensable y no constituye una optimización opcional. MSBuild, en modo incremental, omite la recompilación de aquellos proyectos cuyos archivos de entrada no han cambiado respecto de la última compilación exitosa. Dado que los analizadores se ejecutan como parte del compilador, la omisión de un proyecto implica que ningún analizador se ejecuta sobre él, y el resultado es un reporte que aparece incompleto o vacío sin arrojar error alguno durante el proceso.

Este comportamiento silencioso lo convierte en una de las causas más frecuentes de análisis aparentemente exitosos pero sin hallazgos, escenario particularmente engañoso porque el mensaje ANALYSIS SUCCESSFUL se emite con normalidad.

2.3 Comparabilidad entre corridas

Ambos análisis se ejecutaron con parámetros idénticos —misma clave de proyecto, mismo perfil de calidad, mismas rutas de reportes de cobertura y misma versión del scanner— de modo que las diferencias observadas en las métricas son atribuibles exclusivamente a los cambios introducidos en el código y no a variaciones en la configuración del análisis. Este control es metodológicamente necesario para que la comparación antes/después tenga validez.

3. Modelo de clasificación de SonarQube

Para interpretar correctamente los resultados es necesario precisar la taxonomía que la herramienta emplea, ya que la relación entre tipo de issue, calificación afectada y flujo de resolución no es intuitiva y explica varios de los resultados obtenidos.

3.1 Tipos de issue y calificación afectada

Tipo	Dimensión Clean Code	Métrica conteo	de Calificación afectada	Flujo de resolución
Bug	Reliability	bugs	reliability_rating	Corrección obligatoria
Vulnerability	Security	vulnerabilities	security_rating	Corrección obligatoria
Security Hotspot	Security	security_hotspots	security_review_rating	Revisión humana (To Review → Acknowledged / Fixed / Safe)
Code Smell	Maintainability	code_smells	sqale_rating	Corrección según deuda técnica

La distinción entre **Vulnerability** y **Security Hotspot** es determinante para interpretar los resultados de esta práctica. Una vulnerabilidad representa una falla de seguridad respecto de la cual el analizador tiene certeza; un hotspot señala código sensible desde el punto de vista de la seguridad cuya peligrosidad real depende del contexto de uso, y respecto del cual la herramienta no puede pronunciarse por sí sola. Por esa razón un hotspot no exige corrección automática sino revisión humana documentada, y **no afecta el security_rating** sino una métrica independiente denominada `security_review_rating`.

3.2 Escala de calificaciones

Las calificaciones de Reliability y Security se determinan por la severidad del peor issue presente: una calificación A implica ausencia de issues de esa dimensión, mientras que las calificaciones descendentes corresponden a la presencia de al menos un issue de severidad Minor, Major, Critical o Blocker respectivamente.

El `sqale_rating` (Maintainability) opera con una lógica distinta: se calcula a partir del **ratio de deuda técnica**, definido como el cociente entre el esfuerzo estimado de remediación y el esfuerzo estimado de desarrollo del código analizado. Una calificación A corresponde a un ratio igual o inferior al 5 %, umbral que un proyecto

de reducido tamaño puede satisfacer aun conteniendo varios code smells de baja severidad.

3.3 Criterio de evaluación del Quality Gate

El Quality Gate predeterminado, denominado *Sonar way*, evalúa sus condiciones **sobre el código nuevo o modificado**, no sobre la totalidad del proyecto. Este criterio, conocido como *Clean as You Code*, evita que la deuda técnica heredada bloquee de manera permanente los despliegues, mientras exige que todo código nuevo cumpla el estándar establecido. Su implicación práctica es que la puerta de calidad puede reportar estado OK aun cuando el proyecto conserve hallazgos históricos sin resolver.

4. Resumen ejecutivo — Comparativo antes / después

Dimensión	Métrica SonarQube	Antes	Después	Variación
Reliability	bugs	0	0	Sin cambio
Reliability	reliability_rating	A (1.0)	A (1.0)	Sin cambio
Security	vulnerabilities	0	0	Sin cambio
Security	security_hotspots pendientes	1	0	-1 (-100 %)
Security	security_rating	A (1.0)	A (1.0)	Sin cambio
Maintainability	code_smells	8	0	-8 (-100 %)
Maintainability	sqale_rating	A (1.0)	A (1.0)	Sin cambio
Cobertura	coverage	68.9 %	72.0 %	+3.1 pp
Duplicación	uplicated_lines_density	0.0 %	0.0 %	Sin cambio
Volumen	ncloc	151	150	-1 línea
Puerta de calidad	Quality Gate	OK	OK	Estable

4.1 Interpretación de las calificaciones invariantes

Un resultado que requiere explicación explícita, por resultar contraintuitivo, es que las tres calificaciones permanecieran en A desde el primer análisis pese a la existencia de nueve hallazgos.

La causa se encuentra en el mecanismo descrito en la sección 3. El `security_rating` únicamente desciende ante la presencia de issues de tipo **Vulnerability**; el hallazgo de inyección SQL se clasificó como **Security Hotspot**, categoría que por diseño no impacta esa calificación sino el `security_review_rating`, precisamente porque su explotabilidad requiere confirmación humana. De manera análoga, el `reliability_rating` solo desciende ante issues de tipo **Bug**, y ninguno fue reportado como tal. Finalmente, el `sqale_rating` se mantuvo en A porque los ocho code smells, dado el reducido volumen del proyecto (151 líneas de código), generaron un ratio de deuda técnica inferior al umbral del 5 %.

En consecuencia, **las calificaciones agregadas no constituyen un indicador suficiente del estado de un proyecto**, y el análisis debe apoyarse en los conteos absolutos de issues y en la revisión individual de cada hallazgo. Esta observación tiene valor formativo por sí misma: un tablero completamente en verde no equivale a un proyecto libre de hallazgos.

5. Detalle de los issues resueltos

5.1 Security — Security Hotspot (1 → 0)

Hallazgo: construcción dinámica de consulta SQL mediante concatenación

Atributo	Valor
Categoría de seguridad	sql-injection
Ubicación	MiAppNetCore/Services/WeatherService.cs:73
Nivel de riesgo (vulnerabilityProbability)	HIGH
Clasificación OWASP	A03:2021 — Injection
Clasificación CWE	CWE-89 — <i>Improper Neutralization of Special Elements used in an SQL Command</i>
Estado inicial	To Review
Estado tras revisión	Acknowledged, con comentario justificativo
Estado tras remediación	No detectado (0 hotspots)

Descripción técnica del defecto. La consulta se construía incorporando la entrada del usuario directamente al texto de la sentencia mediante concatenación de

cadenas, siguiendo el patrón "...LIKE '%" + cityName + "%'". Este patrón mezcla dos canales que deben permanecer estrictamente separados: el canal de instrucciones y el canal de datos. El motor de base de datos recibe una única cadena que analiza sintácticamente en su totalidad, sin disponer de mecanismo alguno para distinguir qué fragmento fue escrito por el desarrollador y cuál provino de una entrada externa. Los caracteres que el desarrollador consideraba simples datos —comillas simples, dobles guiones, punto y coma— son en realidad símbolos con significado sintáctico, de modo que su introducción permite reescribir la lógica de la consulta.

Vectores de explotación aplicables al patrón detectado. Una comilla simple cierra prematuramente el literal de texto, habilitando la inyección de sintaxis arbitraria a continuación. A partir de ahí, una condición tautológica del tipo OR '1'='1 neutraliza el filtro y provoca la devolución del conjunto completo de registros. La cláusula UNION SELECT permite extraer datos de tablas distintas a la consultada, incluidas las tablas de metadatos que exponen la estructura completa del esquema. En el caso específico de SQLite, `sqlite_master` contiene las sentencias CREATE de todos los objetos de la base de datos. Adicionalmente, mediante técnicas ciegas basadas en diferencias de respuesta resulta factible reconstruir el contenido de la base de datos incluso cuando la aplicación no expone resultados al usuario.

Corrección aplicada. Se substituyó la concatenación por un parámetro real:

```
csharp
```

```
command.Parameters.AddWithValue("$namePattern", $"%{cityName}%");
```

Fundamento de la corrección. Con la parametrización, el proveedor de datos transmite la sentencia y los valores por canales independientes del protocolo. El motor recibe primero la estructura de la consulta, la compila y fija su plan de ejecución, y solo entonces recibe los valores, que ya no pueden alterar un plan previamente establecido. Un payload de inyección se busca entonces de forma literal, como si fuera un nombre de ciudad ordinario, y el resultado es simplemente que no existe coincidencia.

Debe destacarse un detalle de diseño relevante en esta corrección: los comodines % permanecen dentro del **valor** del parámetro y no en el texto de la sentencia. Esta es la construcción correcta, dado que los comodines de LIKE pertenecen conceptualmente al dato de búsqueda y no a la sintaxis de la consulta. La alternativa de mantenerlos en el texto SQL concatenando únicamente la variable reintroduciría parcialmente el problema.

Consideraciones adicionales de robustez. Aun con la parametrización aplicada, subsisten dos aspectos que en un contexto de producción merecerían atención. En primer lugar, los caracteres % y _ introducidos por el usuario conservan su

significado de comodín dentro del patrón, por lo que una entrada consistente en un único % produce un escaneo completo de la tabla; la mitigación consiste en escapar dichos caracteres mediante la cláusula ESCAPE. En segundo lugar, AddWithValue infiere el tipo del parámetro a partir del valor de .NET, comportamiento aceptable en SQLite dada su tipificación dinámica, pero que en proveedores como SQL Server puede inducir conversiones implícitas que invalidan el uso de índices; la práctica recomendada en ese contexto es declarar el tipo de forma explícita mediante Add con el DbType correspondiente.

Gestión del hallazgo en la herramienta. El hotspot se marcó con estado *Acknowledged* acompañado de un comentario que documenta la corrección aplicada, replicando el flujo de trabajo profesional en el cual el hallazgo deja de figurar como pendiente y queda registrado como una decisión evaluada, con trazabilidad sobre quién la tomó y bajo qué justificación. Tras la ejecución del nuevo análisis, el hotspot dejó de detectarse.

5.2 Maintainability — Code Smells (8 → 0)

# Regla	Ubicación	Severidad	Naturaleza del defecto
1 external_roslyn:CS8602	WeatherService.cs GetCityWeatherSummary	— MAJOR	Dereferencia de referencia potencialmente nula
2 csharpsquid:S1481 external_roslyn:CS0219	/ WeatherService.cs:112 — GetWeatherDescription	MINOR MAJOR	/ Variable local declarada y nunca utilizada
3 csharpsquid:S2325	WeatherService.cs:96 IsValidTemperature	— MINOR	Método sin acceso a estado de instancia no declarado static
4 csharpsquid:S2325	WeatherService.cs:110 — GetWeatherDescription	MINOR	Idéntico al anterior

# Regla	Ubicación	Severidad	Naturaleza del defecto
5 csharpsquid:S6966	Program.cs:24	MAJOR	Uso de la variante síncrona Run() en lugar de RunAsync()
6 csharpsquid:S1118	Program.cs:26	MAJOR	Clase sin miembros de instancia carente de constructor no público
7 external_roslyn:ASP0027	Program.cs:26	INFO	Declaración de public partial class Program innecesaria en el SDK actual

Hallazgo 1 — Dereferencia de referencia potencialmente nula (CS8602)

Descripción. Se accedía a la propiedad match.Name sin comprobar previamente si la operación de búsqueda había devuelto un resultado. El compilador, con el análisis de tipos de referencia anulables habilitado, determina mediante análisis de flujo que existe al menos una ruta de ejecución alcanzable en la cual match conserva el valor nulo en el punto de la dereferencia.

Impacto. Ante una búsqueda sin coincidencias, la aplicación lanza una `NullReferenceException` no controlada. En el contexto de una aplicación web esto se traduce en una respuesta HTTP 500 que, según la configuración del entorno, puede exponer la traza de pila completa al cliente, revelando rutas del sistema de archivos, nombres de ensamblados y estructura interna del código. Corresponde conceptualmente a *CWE-476 (NULL Pointer Dereference)*, y su explotación deliberada mediante entradas que no producen coincidencia constituye un vector de denegación de servicio de bajo costo.

Corrección aplicada. Se incorporó una validación explícita previa al uso del resultado:

```
csharp  
if (match is null)
```

```
    return "No se encontraron resultados...";
```

Observación sobre la clasificación. Este hallazgo se importó bajo el tipo **Code Smell** por tratarse de un diagnóstico externo del compilador (prefijo `external_roslyn`), no de una regla nativa del analizador de SonarQube. Los diagnósticos importados desde Roslyn se clasifican por defecto en la dimensión de mantenibilidad, con independencia de que su naturaleza conceptual corresponda a la de un defecto de fiabilidad. Esta distinción explica que el conteo de bugs permaneciera en cero pese a la presencia de un riesgo de referencia nula, y constituye un ejemplo concreto de por qué la lectura de las métricas agregadas debe complementarse con la revisión individual de los hallazgos.

Hallazgo 2 — Variable local sin uso (S1481 / CS0219)

Descripción. La variable `unitLabel` se declaraba e inicializaba sin que su valor fuera leído en ninguna ruta posterior. El doble reporte —por la regla nativa de SonarQube y por el diagnóstico del compilador— responde a que ambos analizadores detectan la condición de manera independiente.

Impacto. Aunque el compilador elimina la asignación durante la optimización y el defecto no altera el comportamiento en tiempo de ejecución, su presencia incrementa la carga cognitiva de quien mantiene el código y con frecuencia constituye evidencia de una operación prevista que quedó incompleta, situación que puede encubrir un defecto funcional real. Se relaciona con CWE-563 (*Assignment to Variable without Use*).

Corrección aplicada. Eliminación de la declaración. Este cambio explica la reducción de una línea en la métrica `ncloc` (151 → 150).

Hallazgos 3 y 4 — Métodos sin acceso a estado de instancia (S2325)

Descripción. Los métodos `IsValidTemperature` y `GetWeatherDescription` no acceden a ningún campo, propiedad ni método de instancia de la clase que los contiene, por lo que su comportamiento resulta independiente del estado del objeto.

Impacto. Más allá del aspecto puramente estilístico, la declaración `static` transmite una garantía verificable por el compilador: el método no puede depender del estado de la instancia ni modificarlo. Esto simplifica el razonamiento sobre el código, elimina la necesidad de construir una instancia para ejercitarlo en pruebas unitarias y previene la introducción accidental de acoplamiento con el estado en modificaciones futuras. Adicionalmente, la invocación de métodos estáticos evita el paso del puntero implícito `this` y la resolución de despacho virtual.

Corrección aplicada. Se declararon ambos métodos como static.

Hallazgo 5 — Uso de la variante síncrona de arranque (S6966)

Descripción. El punto de entrada de la aplicación invocaba `app.Run()`, variante que bloquea el hilo principal hasta la terminación del host, en lugar de `app.RunAsync()`.

Impacto. La variante síncrona provoca el bloqueo del hilo llamante durante toda la vida de la aplicación. En un modelo de ejecución asíncrono, bloquear un hilo del pool para esperar una operación de larga duración impide su reutilización y, en escenarios de mayor complejidad, puede derivar en agotamiento del pool de hilos. La versión asíncrona permite además propagar correctamente los tokens de cancelación y participar de forma ordenada en el apagado controlado del host.

Corrección aplicada. Sustitución por `await app.RunAsync();`, aprovechando el soporte de punto de entrada asíncrono en el modelo de sentencias de nivel superior de C#.

Hallazgos 6 y 7 — Declaración innecesaria de Program (S1118, ASP0027)

Descripción. El proyecto declaraba `public partial class Program { }`, construcción utilizada por convención para exponer el tipo `Program` a las pruebas de integración que emplean `WebApplicationFactory<Program>`. La regla S1118 señalaba que una clase carente de miembros de instancia debería declarar un constructor `protected` o `static` a fin de impedir su instanciación, mientras que ASP0027 advertía que el SDK actual de ASP.NET Core genera automáticamente esa declaración, tornando la explícita redundante.

Decisión de remediación y su justificación. Se evaluó que la suite de pruebas del proyecto ejercita `WeatherService` de forma directa mediante pruebas unitarias, sin recurrir a `WebApplicationFactory<Program>` ni a ningún otro mecanismo de prueba de integración sobre el host. En consecuencia, la declaración no cumplía función alguna y **se eliminó por completo**, resolviendo ambos hallazgos con un único cambio.

Esta decisión ilustra un principio de remediación que conviene explicitar: ante un hallazgo caben al menos tres respuestas —satisfacer literalmente la recomendación de la regla, eliminar el código que la origina, o justificar su permanencia marcándola como no aplicable. La segunda opción resulta preferible cuando el código señalado no aporta valor, dado que suprime el hallazgo y reduce simultáneamente la superficie de mantenimiento, en lugar de añadir construcciones adicionales para satisfacer una regla sobre código innecesario.

5.3 Reliability — Bugs (0 → 0)

No se generaron issues de tipo **Bug** en ninguno de los dos análisis, y el `reliability_rating` se mantuvo en A en ambas corridas.

Se deja constancia explícita de esta categoría para que el reporte no la omita, y se reitera la precisión formulada en el hallazgo 1 de la sección 5.2: el riesgo de dereferencia nula (CS8602), que conceptualmente pertenece a la dimensión de fiabilidad, fue importado por SonarQube bajo el tipo **Code Smell** por tratarse de un diagnóstico externo de Roslyn. Su corrección se documenta por tanto en la sección de mantenibilidad, si bien su efecto real corresponde a una mejora de la fiabilidad del componente.

6. Verificación mediante la API del servidor

La validación del estado final no se realizó por inspección visual del dashboard, sino mediante consulta directa a la API REST, procedimiento que elimina la posibilidad de error de lectura y produce evidencia reproducible.

6.1 Consulta de métricas agregadas

GET `/api/measures/component`

`?component=MiAppNetCore`

`&metricKeys=security_rating,reliability_rating,sqale_rating,
bugs,vulnerabilities,code_smells,security_hotspots,
coverage,duplicated_lines_density,ncloc`

Respuesta obtenida (valores extraídos):

json

```
{  
  "bugs": "0",  
  "vulnerabilities": "0",  
  "code_smells": "0",  
  "security_hotspots": "0",  
  "security_rating": "1.0",  
  "reliability_rating": "1.0",  
  "sqale_rating": "1.0",  
  "coverage": "72.0",  
  "duplicated_lines_density": "0.0",  
  "ncloc": "150"
```

}

Los valores 1.0 corresponden a la calificación A en la escala interna de la herramienta, donde el rango va de 1.0 (A) a 5.0 (E).

6.2 Consulta del estado del Quality Gate

GET /api/qualitygates/project_status?projectKey=MiAppNetCore

→ "status": "OK"

6.3 Resultados de la suite de pruebas

La ejecución de dotnet test reportó **15 de 15 pruebas exitosas**, sin fallos ni pruebas omitidas. La cobertura se recolectó mediante coverlet.collector en formato OpenCover y fue importada por el sensor de cobertura de SonarQube durante la fase end, quedando correlacionada por archivo y por línea con los resultados del análisis estático.

7. Análisis de la evolución de la cobertura

La cobertura registró un incremento de 68.9 % a 72.0 %, equivalente a 3.1 puntos porcentuales. Este comportamiento merece interpretación, dado que la remediación de hallazgos no incrementa la cobertura de manera automática.

El aumento responde a dos factores concurrentes. Por una parte, las correcciones se acompañaron de ajustes y adiciones en la suite de pruebas de MiAppNetCore.Tests, orientados a verificar el comportamiento resultante de cada cambio —de manera destacada, la validación de resultado nulo incorporada en GetCityWeatherSummary, cuya rama alternativa requiere una prueba específica para ser ejercitada. Por otra parte, la eliminación de la variable sin uso y de la declaración redundante de Program redujo el denominador de la métrica en una línea, dado que se trataba de código no cubierto por prueba alguna.

Este segundo factor ilustra una propiedad relevante de la métrica que conviene tener presente: **la cobertura puede incrementarse tanto añadiendo pruebas como eliminando código no probado**. Ambos movimientos son legítimos, pero solo el primero incrementa efectivamente la verificación del comportamiento del sistema, razón por la cual la métrica debe leerse siempre junto con la evolución del volumen de código analizado (ncloc), y no de forma aislada.

Corresponde precisar, adicionalmente, que la cobertura mide ejecución y no verificación: una prueba que invoca un método sin evaluar aserción alguna incrementa la métrica sin aportar garantía sobre la corrección del comportamiento. Del mismo modo, una cobertura del cien por ciento no implicaría ausencia de vulnerabilidades, puesto que las pruebas convencionales verifican el comportamiento previsto y no el comportamiento adversarial.

8. Hallazgos fuera del alcance del análisis estático

Durante la compilación, dotnet build reportó **dos advertencias NU1903**, correspondientes a vulnerabilidades conocidas y publicadas en paquetes NuGet de dependencia transitiva:

Paquete	Versión	Naturaleza
Microsoft.OpenApi	2.0.0	Dependencia transitiva con vulnerabilidad conocida publicada
SQLitePCLRaw.lib.e_sqlite3	2.1.11	Dependencia transitiva nativa con vulnerabilidad conocida publicada

Estas advertencias no fueron generadas por SonarQube, sino por el propio cliente de NuGet al contrastar el grafo de dependencias resuelto contra la base de datos de avisos de seguridad de GitHub. SonarQube Community Edition no incorpora análisis de composición de software (SCA) de manera predeterminada, disciplina que examina las dependencias de terceros en lugar del código propio.

La distinción es conceptualmente importante y constituye, en sí misma, un hallazgo formativo de la práctica. El análisis estático de código (SAST) y el análisis de composición de software (SCA) cubren superficies de ataque disjuntas: el primero examina el código escrito por el equipo, el segundo el código incorporado desde terceros. Un proyecto puede alcanzar cero issues en SAST y encontrarse simultáneamente expuesto a través de una dependencia transitiva vulnerable, escenario que las estadísticas de la industria señalan como origen de una proporción mayoritaria del código presente en una aplicación moderna.

Las dependencias transitivas presentan además una dificultad de remediación específica: no figuran de forma explícita en el archivo de proyecto, sino que se incorporan como requisitos de otros paquetes, por lo que su actualización exige elevar la versión del paquete padre o forzar la resolución mediante una referencia directa a la versión corregida.

Estas advertencias quedan por tanto **fuera del alcance del criterio de cero issues** verificado en este reporte, que se refiere exclusivamente a los hallazgos del análisis estático, y se documentan aquí por su relevancia para la postura de seguridad global del proyecto.

9. Limitaciones del análisis realizado

En reconocimiento de los límites metodológicos del ejercicio, se consignan las siguientes precisiones:

- **Alcance del analizador.** El análisis estático detecta patrones asociados a defectos conocidos; no identifica errores en la lógica de negocio, fallas de diseño en el modelo de autorización ni vulnerabilidades derivadas de la interacción entre componentes.
- **Ausencia de análisis dinámico.** No se ejecutaron pruebas de penetración ni escaneo dinámico (DAST), disciplinas que detectan condiciones observables únicamente con la aplicación en ejecución.
- **Ausencia de análisis de composición.** Conforme a lo expuesto en la sección 8, las dependencias de terceros no fueron evaluadas por la herramienta.
- **Perfil de calidad predeterminado.** El análisis se ejecutó con el perfil *Sonar way*, que no incorpora la totalidad de las reglas disponibles. Un perfil personalizado con reglas adicionales activadas produciría un conjunto de hallazgos más amplio.
- **Volumen del proyecto.** Con 150 líneas de código analizadas, las métricas relativas —de manera señalada el ratio de deuda técnica y la densidad de duplicación— presentan una sensibilidad elevada a cambios pequeños y no resultan extrapolables a proyectos de escala productiva.

10. Conclusiones

Se resolvieron la totalidad de los issues detectados por SonarQube en las tres dimensiones del modelo Clean Code: un Security Hotspot correspondiente a inyección SQL y ocho Code Smells, entre los que se incluye el riesgo de dereferencia de referencia nula. Tras aplicar las correcciones y ejecutar nuevamente el ciclo completo de análisis, el proyecto quedó en **0 Bugs, 0 Vulnerabilities, 0 Code Smells y 0 Security Hotspots**, con calificación **A** en Security, Reliability y Maintainability, y **Quality Gate en estado OK**, verificado mediante consulta directa a la API del servidor.

La práctica evidencia que la lectura de las calificaciones agregadas resulta insuficiente como criterio de evaluación. Las tres calificaciones se encontraban en A desde el primer análisis pese a la existencia de nueve hallazgos, incluida una condición de inyección SQL con nivel de riesgo HIGH. Este comportamiento no constituye un defecto de la herramienta sino una consecuencia directa de su modelo

de clasificación, y su comprensión resulta indispensable para interpretar correctamente cualquier reporte de análisis estático.

La distinción entre Security Hotspot y Vulnerability introduce una idea de fondo relevante para la disciplina: la seguridad no admite reducirse a una evaluación binaria. Existe una franja amplia de código sensible cuyo riesgo depende del contexto de uso, y esa franja exige revisión humana documentada antes que automatización. El flujo de estados de los hotspots materializa ese principio dentro de la herramienta, y su uso correcto —revisar, decidir y dejar constancia— constituye parte integral de la competencia que la práctica busca desarrollar.

La cobertura de pruebas se mantuvo estable y en ligero ascenso porque las correcciones se acompañaron de la extensión de la suite, evitando que los cambios introdujeran regresiones no detectadas. Este acoplamiento entre remediación y prueba resulta esencial: corregir sin verificar traslada el riesgo en lugar de eliminarlo.

Finalmente, la aparición de advertencias NU1903 durante la compilación, ajenas al análisis estático, demuestra que la seguridad de una aplicación no se agota en la calidad del código propio. Un proyecto con cero issues en SAST puede permanecer expuesto a través de su cadena de suministro de dependencias, lo que justifica la incorporación de análisis de composición de software como control complementario y no sustitutivo.

11. Referencias

- SonarSource. *SonarQube Documentation* — Clean Code taxonomy, Security Hotspots, Quality Gates, Metric Definitions.
- SonarSource. *SonarScanner for .NET* — Documentación de instalación, configuración y parámetros de análisis.
- SonarSource. *Web API Documentation* — Endpoints /api/measures, /api/issues, /api/hotspots, /api/qualitygates.
- OWASP. *OWASP Top 10:2021* — A03 Injection; A06 Vulnerable and Outdated Components.
- OWASP. *Cheat Sheet Series* — SQL Injection Prevention Cheat Sheet.
- MITRE. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command*.
- MITRE. *CWE-476: NULL Pointer Dereference*.
- MITRE. *CWE-563: Assignment to Variable without Use*.
- Microsoft Learn. *Microsoft.Data.Sqlite* — Parameterized queries.
- Microsoft Learn. *Nullable reference types* y diagnóstico CS8602.
- Microsoft Learn. *NuGet Warning NU1903* — Package with known vulnerability.
- .NET Foundation. *Roslyn Analyzers* — Arquitectura de analizadores y diagnósticos externos.